

1. AI and Liability

Source: Simon Willison's Weblog | Published: 2026-06-25T22:28:46+00:00

Link: <https://simonwillison.net/2026/Jun/25/ai-and-liability/#atom-everything>

AI and Liability Bruce Schneier on the recent German ruling that Google be held liable for errors introduced in their AI overviews: AI agents are agents of the person or organization that deploys them?and should be treated by the law as such. If a company hired human writers to write its summaries, that company would be liable for inaccuracies in those summaries. [...] To allow businesses to hide behind the excuse of faulty AI in those same circumstances would be a massive handout to companies, and would introduce disastrous incentives for corporate misbehavior. Why hire human writers, lawyers or doctors when AIs are not only cheaper, but also absolve employers whenever they make a mistake? Tags: bruce-schneier , google , law , ai , generative-ai , llms , ai-ethics , hallucinations

2. datasette-export-database 0.3a2

Source: Simon Willison's Weblog | Published: 2026-06-25T17:21:09+00:00

Link: <https://simonwillison.net/2026/Jun/25/datasette-export-database/#atom-everything>

Release: datasette-export-database 0.3a2 An embarrassingly tiny release. The pyproject.toml had pinned to datasette==1.0a27 , inadvertently making this plugin incompatible with all other Datasette versions. It's now datasette>=1.0a27 instead. Tags: datasette

3. Raymond's hot take on Hainanese chicken

Source: The Old New Thing | Published: Thu, 25 Jun 2026 14:00:01 +0000

Link: <https://devblogs.microsoft.com/oldnewthing/20260625-01/?p=112469>

Subtlety. The post Raymond's hot take on Hainanese chicken appeared first on The Old New Thing .

4. The case of the DLL that was not present in memory despite not being formally unloaded, part 1

Source: The Old New Thing | Published: Thu, 25 Jun 2026 14:00:00 +0000

Link: <https://devblogs.microsoft.com/oldnewthing/20260625-00/?p=112467>

Figuring out how it went missing. The post The case of the DLL that was not present in memory despite not being formally unloaded, part 1 appeared first on The Old New Thing .

5. Cancellation of Windows Runtime activities is asynchronous

Source: The Old New Thing | Published: Wed, 24 Jun 2026 14:00:00 +0000

Link: <https://devblogs.microsoft.com/oldnewthing/20260624-00/?p=112465>

You're asking for it to cancel, but it doesn't wait for confirmation. The post Cancellation of Windows Runtime activities is asynchronous appeared first on The Old New Thing .

6. Apple Journal's Atrocious Undo Bug Has Been Fixed (and SwiftUI, Per Se, Is Not to Blame)

Source: Daring Fireball | Published: 2026-06-25T22:49:32Z

Link: https://daringfireball.net/2026/06/swiftui_only_makes_it_easy_to_develop_bad_apps

On the eve of WWDC, in a post arguing that ? SwiftUI Only Makes It Easy to Develop Bad Apps ?, I wrote about an atrocious bug in Apple's Journal app: If you're running macOS 26 Tahoe, open Journal and make a new dummy entry. Type something like ?The quick brown fox.? Then double-click on the word ?brown? and delete it. Now invoke Undo. What you expect is for the word ?brown? to reappear. What happens is ... the whole sentence disappears . Gone. Invoke Redo and you only get back to ?The quick fox.? The word ?brown? is just gone forever. It's nowhere in the Undo stack. That's just profoundly fucked up. I've never seen anything like this with an AppKit app, ever. (I've never seen it with a UIKit app either ? and the same thing happens on iOS with Journal. It's just that you notice it less often because we don't invoke Undo and Redo

nearly as often there.) Marcin Wichary, linking to my post from his remarkably good, remarkably prolific blog [Unsung](#) , wrote: Software engineering typically has some categories of bugs and failures that result in immediate action ? a night shift, a war room, ?sevs,? and so on. Those are, in my experience, things like: the app crashes, the site doesn't load...

7. ? Spensive Thoughts

Source: [Daring Fireball](#) | Published: 2026-06-25T22:36:09Z

Link: https://daringfireball.net/2026/06/spensive_thoughts

Some quick thoughts on the hardware prices Apple increased ? and didn't increase ? today.

8. Om Malik, 1966-2026

Source: [Daring Fireball](#) | Published: 2026-06-25T20:31:37Z

Link: <https://om.co/2026/06/24/1966-2026/>

Heartbreaking news, shared by Om's family: Om Malik passed away on June 24, 2026, at Stanford Hospital after a long health journey with his heart. He was surrounded by family and friends.

We invite you to share your remembrances of Om in the comments below or by posting and tagging his accounts on X/Twitter , Instagram , Threads , or LinkedIn . Om kept this battle very private, so this news comes as a terrible surprise for many, and an incomprehensible gut punch for everyone who knew and loved him. Rest in peace, my friend. So it goes. ?

9. Some things on ' systemctl kexec ' as compared to ' kexec -e '

Source: [Chris's Wiki :: blog](#) | Published: 2026-06-25T02:43:01Z

Link: <https://utcc.utoronto.ca/~cks/space/blog/linux/SystemctlKexecVsKexec>

Suppose, unfortunately not hypothetically, that you have some machines that the Ubuntu 26.04 LTS installer kernel sometimes gets a kernel oops during a network based reinstall (we don't think this is a hardware flaw, but who knows; these machines were stable on 24.04). Further suppose that you're not network booting these machines but instead you're using kexec to boot them into the installer environment . This creates an awkward situation, where the over the network installer may have gotten far enough before the panic to have written over enough of the disk (with the previous install on it) so you can't reboot from it. Fortunately there is a way out, because you can contrive to kexec the installer environment again from within the installer environment. (A kernel oops is not a kernel panic, and normally doesn't reboot the system unless you've made specific settings changes , which the Ubuntu 26.04 LTS installer environment hasn't done. In this case, that's a good thing for us.) Once you've used ' kexec -l ' to load the installer kernel and initrd with the required command line arguments, you have two choices for actually rebooting into the kexec kernel. You can use the plain and...

10. Hart's theorem

Source: [John D. Cook](#) | Published: Thu, 25 Jun 2026 19:20:09 +0000

Link: <https://www.johndcook.com/blog/2026/06/25/harts-theorem/>

Hart's theorem says If a triangle be formed by the arcs of three circles, the inscribed and the three escribed circles are all tangent to a new circle or line. Here ?triangle? means a three-sided figure whose sides are portions of a circle. The inscribed circle is the largest circle that can fit inside the [?] The post Hart's theorem first appeared on [John D. Cook](#) .

11. Incircles and Excircles of Pythagorean triangles

Source: [John D. Cook](#) | Published: Thu, 25 Jun 2026 14:35:52 +0000

Link: <https://www.johndcook.com/blog/2026/06/25/incircle-excircle/>

This post will reveal the connection between my two previous posts: one on the Star Trek lemma and one on Pythagorean triples. In the process of writing the latter, I looked at the Wikipedia article on Pythagorean triples and noticed this curious paragraph. In every Pythagorean triangle, the radius of the incircle and the radii of the [?] The post Incircles and Excircles of Pythagorean triangles first appeared on [John D. Cook](#) .

12. Consecutive Pythagorean triangle sides

Source: John D. Cook | Published: Thu, 25 Jun 2026 12:00:14 +0000

Link: <https://www.johndcook.com/blog/2026/06/25/consecutive-pythagorean/>

In this post we find all Pythagorean triples that contain consecutive numbers, all Pythagorean triples (a, b, c) such that $a + 1 = b$ or $b + 1 = c$. $a + 1 = b$ George Osborne wrote a paper [1] addressing the question of when the squares of two consecutive numbers is also a square. Geometrically this is asking [?] The post Consecutive Pythagorean triangle sides first appeared on John D. Cook .

13. "No way to prevent this" say users of only language where this regularly happens

Source: Xe Iaso's blog | Published: Thu, 25 Jun 2026 00:00:00 GMT

Link: <https://xeiaso.net/shitposts/no-way-to-prevent-this/memory-safety/CVE-2026-8461/>

In the hours following the release of CVE-2026-8461 for the project FFmpeg , site reliability workers and systems administrators scrambled to desperately rebuild and patch all their systems to fix an out-of-bounds write in the MagicYUV decoder (libavcodec/magicyuv.c) caused by improper bounds checking, resulting in heap corruption, denial of service, and potential remote code execution when processing a specially crafted video file. This is due to the affected components being written in C, the only programming language where these vulnerabilities regularly happen. "This was a terrible tragedy, but sometimes these things just happen and there's nothing anyone can do to stop them," said programmer Mrs. Kitty Smitham, echoing statements expressed by hundreds of thousands of programmers who use the only language where 90% of the world's memory safety vulnerabilities have occurred in the last 50 years, and whose projects are 20 times more likely to have security vulnerabilities. "It's a shame, but what can we do? There really isn't anything we can do to prevent memory safety vulnerabilities from happening if the programmer doesn't want to write their code in a robust manner." At press...

14. US Subways Build Too Many Cross Passages

Source: Construction Physics | Published: Thu, 25 Jun 2026 13:55:49 GMT

Link: <https://www.construction-physics.com/p/us-subways-build-too-many-cross-passages>

I wrote the following piece for IFP's Transit Abundance Playbook, a collection of 15 ideas to improve transit delivery in the US.

15. Scrutineer: scanning open source without flooding maintainers

Source: Andrew Nesbitt | Published: 2026-06-25T10:00:00+00:00

Link: <https://nesbitt.io/2026/06/25/scrutineer.html>

Scrutineer scans open source repositories for security vulnerabilities and then handles everything that follows: verifying each one, working out who to contact, drafting a fix, and tracking it through to a published advisory. I've been building it for Alpha-Omega for the past couple of months. Large language models have made finding vulnerabilities in open source code much easier. Point one at a codebase and it turns up real bugs alongside invented ones, faster and cheaper than the fuzzers and scanners that came before, but the bottleneck hasn't moved with it. Every finding still has to be read, confirmed, and fixed by a maintainer, whose time and attention is a finite resource the whole ecosystem depends on. Trying to secure everything by firing machine-generated reports at maintainers would burn out the people the effort relies on. When I pointed a couple of AI scanners at curl back in May, most of the output collapsed against the project's own disclosure policy, and the findings worth having were buried in the rest. Scrutineer is built so the volume a model can generate never lands directly on a maintainer. You add a repo by URL, it runs a pipeline of skills against the code, a...

16. Pluralistic: Jailbreaking isn't theft (25 Jun 2026)

Source: Pluralistic: Daily links from Cory Doctorow | Published: Thu, 25 Jun 2026 09:37:35 +0000

Link: <https://pluralistic.net/2026/06/25/thieve-different/>

Today's links Jailbreaking isn't theft: It wasn't progress when they did it, it's not piracy when we do it back to them. Hey look at this: Delights to delectate. Object permanence: Major AI breakthrough; Disney v Pooh tombstone; Vancouver riot kiss; Farage admits Brexit lies;

Protecting the web from its founders; Sanders x Hillary; Surveillance pricing v your dollars.
Upcoming appearances: Philadelphia, Chicago, London, Edinburgh, Sydney, Melbourne, Brighton, London, South Bend. Recent appearances: Where I've been. Latest books: You keep readin' em, I'll keep writin' 'em. Upcoming books: Like I said, I'll keep writin' 'em. Colophon: All the rest. Jailbreaking isn't theft ([permalink](#)) It's not often that someone on a panel says something that makes my jaw drop, but that's what happened earlier this week when the moderator of a panel I was on in Toronto described jailbreaking an iPhone as "rampant theft of IP." Some context: the panel was in Toronto, and the nominal subject was "digital sovereignty," though all the panelists (except me) interpreted that to mean "sovereign AI." All of their interventions were focused on how Canada could build and operate its own AI, which I found very...